

$c(J_i, t) = p_i$. For $k = 1, 2, \dots, n$ and $i = 1, 2, \dots, m$, if $C_k \in R_i$, then G contains an edge (C_k, J_i) with capacity $c(C_k, J_i) = \infty$.

- a. Show that if $J_i \in T$ for a finite-capacity cut (S, T) of G , then $C_k \in T$ for each $C_k \in R_i$.
- b. Show how to determine the maximum net revenue from the capacity of a minimum cut of G and the given p_i values.
- c. Give an efficient algorithm to determine which jobs to accept and which experts to hire. Analyze the running time of your algorithm in terms of m , n , and $r = \sum_{i=1}^m |R_i|$.

24-4 Updating maximum flow

Let $G = (V, E)$ be a flow network with source s , sink t , and integer capacities. Suppose that you are given a maximum flow in G .

- a. Suppose that the capacity of a single edge $(u, v) \in E$ increases by 1. Give an $O(V + E)$ -time algorithm to update the maximum flow.
- b. Suppose that the capacity of a single edge $(u, v) \in E$ decreases by 1. Give an $O(V + E)$ -time algorithm to update the maximum flow.

24-5 Maximum flow by scaling

Let $G = (V, E)$ be a flow network with source s , sink t , and an integer capacity $c(u, v)$ on each edge $(u, v) \in E$. Let $C = \max \{c(u, v) : (u, v) \in E\}$.

- a. Argue that a minimum cut of G has capacity at most $C |E|$.
- b. For a given number K , show how to find an augmenting path of capacity at least K in $O(E)$ time, if such a path exists.

The procedure MAX-FLOW-BY-SCALING appearing on the following page modifies the basic FORD-FULKERSON-METHOD procedure to compute a maximum flow in G .

- c. Argue that MAX-FLOW-BY-SCALING returns a maximum flow.
- d. Show that the capacity of a minimum cut of the residual network G_f is less than $2K |E|$ each time line 4 executes.
- e. Argue that the inner **while** loop of lines 5–6 executes $O(E)$ times for each value of K .

```

MAX-FLOW-BY-SCALING( $G, s, t$ )
1   $C = \max \{c(u, v) : (u, v) \in E\}$ 
2  initialize flow  $f$  to 0
3   $K = 2^{\lfloor \lg C \rfloor}$ 
4  while  $K \geq 1$ 
5      while there exists an augmenting path  $p$  of capacity at least  $K$ 
6          augment flow  $f$  along  $p$ 
7       $K = K/2$ 
8  return  $f$ 

```

- f.* Conclude that MAX-FLOW-BY-SCALING can be implemented so that it runs in $O(E^2 \lg C)$ time.

24-6 Widest augmenting path

The Edmonds-Karp algorithm implements the Ford-Fulkerson algorithm by always choosing a shortest augmenting path in the residual network. Suppose instead that the Ford-Fulkerson algorithm chooses a *widest augmenting path*: an augmenting path with the greatest residual capacity. Assume that $G = (V, E)$ is a flow network with source s and sink t , that all capacities are integer, and that the largest capacity is C . In this problem, you will show that choosing a widest augmenting path results in at most $|E| \ln |f^*|$ augmentations to find a maximum flow f^* .

- a.* Show how to adjust Dijkstra's algorithm to find the widest augmenting path in the residual network.
- b.* Show that a maximum flow in G can be formed by successive flow augmentations along at most $|E|$ paths from s to t .
- c.* Given a flow f , argue that the residual network G_f has an augmenting path p with residual capacity $c_f(p) \geq (|f^*| - |f|)/|E|$.
- d.* Assuming that each augmenting path is a widest augmenting path, let f_i be the flow after augmenting the flow by the i th augmenting path, where f_0 has $f(u, v) = 0$ for all edges (u, v) . Show that $|f^*| - |f_i| \leq |f^*| (1 - 1/|E|)^i$.
- e.* Show that $|f^*| - |f_i| < |f^*| e^{-i/|E|}$.
- f.* Conclude that after the flow is augmented at most $|E| \ln |f^*|$ times, the flow is a maximum flow.

24-7 Global minimum cut

A **global cut** in an undirected graph $G = (V, E)$ is a partition (see page 1156) of V into two nonempty sets V_1 and V_2 . This definition is like the definition of cut that we have used in this chapter, except that we no longer have distinguished vertices s and t . Any edge (u, v) with $u \in V_1$ and $v \in V_2$ is said to **cross** the cut.

We can extend this definition of a cut to a multigraph $G = (V, E)$ (see page 1167), and we denote by $c(u, v)$ the number of edges in the multigraph with endpoints u and v . A global cut in a multigraph is still a partition of the vertices, and the value of a global cut (V_1, V_2) is $c(V_1, V_2) = \sum_{u \in V_1, v \in V_2} c(u, v)$. A solution to the **global-minimum-cut problem** is a cut (V_1, V_2) such that $c(V_1, V_2)$ is minimum. Let $\mu(G)$ denote the value of a global minimum cut in a graph or multigraph G .

- a. Show how to find a global minimum cut of a graph $G = (V, E)$ by solving $\binom{|V|}{2}$ maximum-flow problems, each with a different pair of vertices as the source and sink, and taking the minimum value of the cuts found.
- b. Give an algorithm to find a global minimum cut by solving only $\Theta(V)$ maximum-flow problems. What is the running time of your algorithm?

The remainder of this problem develops an algorithm for the global-minimum-cut problem that does not use any maximum-flow computations. It uses the notion of an edge contraction, defined on page 1168, with one crucial difference. The algorithm maintains a multigraph, so that upon contracting an edge (u, v) , it creates a new vertex x , and for any other vertex $y \in V$, the number of edges between x and y is $c(u, y) + c(v, y)$. The algorithm does not maintain self-loops, and so it sets $c(x, x)$ to 0. Denote by $G/(u, v)$ the multigraph that results from contracting edge (u, v) in multigraph G .

Consider what can happen to the minimum cut when an edge is contracted. Assume that, at all points, the minimum cut in a multigraph G is unique. We'll remove this assumption later.

- c. Show that for any edge (u, v) , we have $\mu(G/(uv)) \leq \mu(G)$. Under what conditions is $\mu(G/(uv)) < \mu(G)$?

Next, you will show that if you pick an edge uniformly at random, the probability that it belongs to the minimum cut is small.

- d. Show that for any multigraph $G = (V, E)$, the value of the global minimum cut is at most the average degree of a vertex: that $\mu(G) \leq 2|E|/|V|$, where $|E|$ denotes the total number of edges in the multigraph.

- e. Using the results from parts (c) and (d), show that, if we pick an edge (u, v) uniformly at random, then the probability that (u, v) belongs to the minimum cut is at most $2/V$.

Consider the algorithm that repeatedly chooses an edge at random and contracts it until the multigraph has exactly two vertices, say u and v . At that point, the multigraph corresponds to a cut in the original graph, with vertex u representing all the nodes in one side of the original graph, and v representing all the vertices on the other side. The number of edges given by $c(u, v)$ corresponds exactly to the number of edges crossing the corresponding cut in the original graph. We call this algorithm the *contraction algorithm*.

- f. Suppose that the contraction algorithm terminates with a multigraph whose only vertices are u and v . Show that $\Pr\{c(u, v) = \mu(G)\} = \Omega\left(1/\binom{|V|}{2}\right)$.
- g. Prove that if the contraction algorithm repeats $\binom{|V|}{2} \ln |V|$ times, then the probability that at least one of the runs returns the minimum cut is at least $1 - 1/|V|$.
- h. Give a detailed implementation of the contraction algorithm that runs in $O(V^2)$ time.
- i. Combine the previous parts and remove the assumption that the minimum cut must be unique, to conclude that running the contraction algorithm $\binom{|V|}{2} \ln |V|$ times yields an algorithm that runs in $O(V^4 \lg V)$ time and returns a minimum cut with probability at least $1 - 1/V$.

Chapter notes

Ahuja, Magnanti, and Orlin [7], Even [137], Lawler [276], Papadimitriou and Steiglitz [353], Tarjan [429], and Williamson [458] are good references for network flows and related algorithms. Schrijver [399] has written an interesting review of historical developments in the field of network flows.

The Ford-Fulkerson method is due to Ford and Fulkerson [149], who originated the formal study of many of the problems in the area of network flow, including the maximum-flow and bipartite-matching problems. Many early implementations of the Ford-Fulkerson method found augmenting paths using breadth-first search. Edmonds and Karp [132], and independently Dinic [119], proved that this strategy yields a polynomial-time algorithm. A related idea, that of using “blocking flows,” was also first developed by Dinic [119].

A class of algorithms known as *push-relabel algorithms*, due to Goldberg [185] and Goldberg and Tarjan [188], takes a different approach from the Ford-Fulkerson

method. Push-relabel algorithms allow flow conservation to be violated at vertices other than the source and sink as they execute. Using an idea first developed by Karzonov [251], they allow a *preflow* in which the flow into a vertex may exceed the flow out of the vertex. Such a vertex is said to be *overflowing*. Initially, every edge leaving the source is filled to capacity, so that all neighbors of the source are overflowing. In a push-relabel algorithm, each vertex is assigned an integer height. An overflowing vertex may push flow to a neighboring vertex to which it has a residual edge provided that it is higher than the neighbor. If all residual edges from an overflowing vertex go to neighbors with equal or greater heights, then the vertex may increase its height. Once all vertices other than the sink are no longer overflowing, the preflow is not only a legal flow, but also a maximum flow.

Goldberg and Tarjan [188] gave an $O(V^3)$ -time algorithm that uses a queue to maintain the set of overflowing vertices, as well as an algorithm that uses dynamic trees to achieve a running time of $O(VE \lg(V^2/E + 2))$. Several other researchers developed improved variants and implementations [9, 10, 15, 86, 87, 255, 358], the fastest of which, by King, Rao, and Tarjan [255], runs in $O(VE \log_{E/(V \lg V)} V)$ time.

Another efficient algorithm for maximum flow, by Goldberg and Rao [187], runs in $O(\min\{V^{2/3}, E^{1/2}\} E \lg(V^2/E + 2) \lg C)$ time, where C is the maximum capacity of any edge. Orlin [350] gave an algorithm in the same spirit as this algorithm that runs in $O(VE + E^{31/16} \lg^2 V)$ time. Combining it with the algorithm of King, Rao, and Tarjan results in an $O(VE)$ -time algorithm.

A different approach to maximum flows and related problems is to use techniques from continuous optimization including electrical flows and interior-point methods. The first breakthrough in this line of work is due to Madry [308], who gave an $\tilde{O}(E^{10/7})$ -time algorithm for unit-capacity maximum flow and bipartite maximum matching. (See Problem 3-6 on page 73 for a definition of $\tilde{O}$.) There has been a series of papers in this area for matchings, maximum flows, and minimum-cost flows. The fastest algorithm to date in this line of work for maximum flow is due to Lee and Sidford [285], taking $\tilde{O}(\sqrt{V} E \lg^{O(1)} C)$ time. If the capacities are not too large, this algorithm is faster than the $O(VE)$ -time algorithm mentioned above. Another algorithm, due to Liu and Sidford [303] runs in $\tilde{O}(E^{11/8} C^{1/4})$ time, where C is the maximum capacity of any edge. This algorithm does not run in polynomial time, but for small enough capacities, it is faster than the previous ones.

In practice, push-relabel algorithms currently dominate algorithms based on augmenting paths, continuous-optimization, and linear programming for the maximum-flow problem [88].

Many real-world problems can be modeled as finding matchings in an undirected graph. For an undirected graph $G = (V, E)$, a *matching* is a subset of edges $M \subseteq E$ such that every vertex in V has at most one incident edge in M .

For example, consider the following scenario. You have one or more positions to fill and several candidates to interview. According to your schedule, you are able to interview candidates at certain time slots. You ask the candidates to indicate the subsets of time slots at which they are available. How can you schedule the interviews so that each time slot has at most one candidate scheduled, while maximizing the number of candidates that you can interview? You can model this scenario as a matching problem on a bipartite graph in which each vertex represents either a candidate or a time slot, with an edge between a candidate and a time slot if the candidate is available then. If an edge is included in the matching, that means you are scheduling a particular candidate for a particular time slot. Your goal is to find a *maximum matching*: a matching of maximum cardinality. One of the authors of this book was faced with exactly this situation when hiring teaching assistants for a large class. He used the Hopcroft-Karp algorithm in Section 25.1 to schedule the interviews.

Another application of matching is the U.S. National Resident Matching Program, in which medical students are matched to hospitals where they will be stationed as medical residents. Each student ranks the hospitals by preference, and each hospital ranks the students. The goal is to assign students to hospitals so that there is never a student and a hospital that both have regrets because the student was not assigned to the hospital, yet each ranked the other higher than who or where they were assigned. This scenario is perhaps the best-known real-world example of the “stable-marriage problem,” which Section 25.2 examines.

Yet another instance where matching comes into play occurs when workers must be assigned to tasks in order to maximize the overall effectiveness of the assignment. For each worker and each task, the worker has some quantified effectiveness

for that task. Assuming that there are equal numbers of workers and tasks, the goal is to find a matching with the maximum total effectiveness. Such a situation is an example of an assignment problem, which Section 25.3 shows how to solve.

The algorithms in this chapter find matchings in *bipartite* graphs. As in Section 24.3, the input is an undirected graph $G = (V, E)$, where $V = L \cup R$, the vertex sets L and R are disjoint, and every edge in E is incident on one vertex in L and one vertex in R . A matching, therefore, matches vertices in L with vertices in R . In some applications, the sets L and R have equal cardinality, and in other applications they need not be the same size.

An undirected graph need not be bipartite for the concept of matching to apply. Matching in general undirected graphs has applications in areas such as scheduling and computational chemistry. It models problems in which you want to pair up entities, represented by vertices. Two vertices are adjacent if they represent compatible entities, and you need to find a large set of compatible pairs. Maximum-matching and maximum-weight matching problems on general graphs can be solved by polynomial-time algorithms whose running times are similar to those for bipartite matching, but the algorithms are significantly more complicated. Exercise 25.2-5 discusses the general version of the stable-marriage problem, known as the “stable-roommates problem.” Although matching applies to general undirected graphs, this chapter deals only with bipartite graphs.

25.1 Maximum bipartite matching (revisited)

Section 24.3 demonstrated one way to find a maximum matching in a bipartite graph, by finding a maximum flow. This section provides a more efficient method, the Hopcroft-Karp algorithm, which runs in $O(\sqrt{V}E)$ time. Figure 25.1(a) shows a matching in an undirected bipartite graph. A vertex that has an incident edge in matching M is *matched* under M , and otherwise, it is *unmatched*. A *maximal matching* is a matching M to which no other edges can be added, that is, for every edge $e \in E - M$, the edge set $M \cup \{e\}$ fails to be a matching. A maximum matching is always maximal, but the reverse does not always hold.

Many algorithms to find maximum matchings, the Hopcroft-Karp algorithm included, work by incrementally increasing the size of a matching. Given a matching M in an undirected graph $G = (V, E)$, an *M -alternating path* is a simple path whose edges alternate between being in M and being in $E - M$. Figure 25.1(b) depicts an *M -augmenting path* (sometimes called an augmenting path with respect to M): an M -alternating path whose first and last edges belong to $E - M$. Since an M -augmenting path contains one more edge in $E - M$ than in M , it must consist of an odd number of edges.

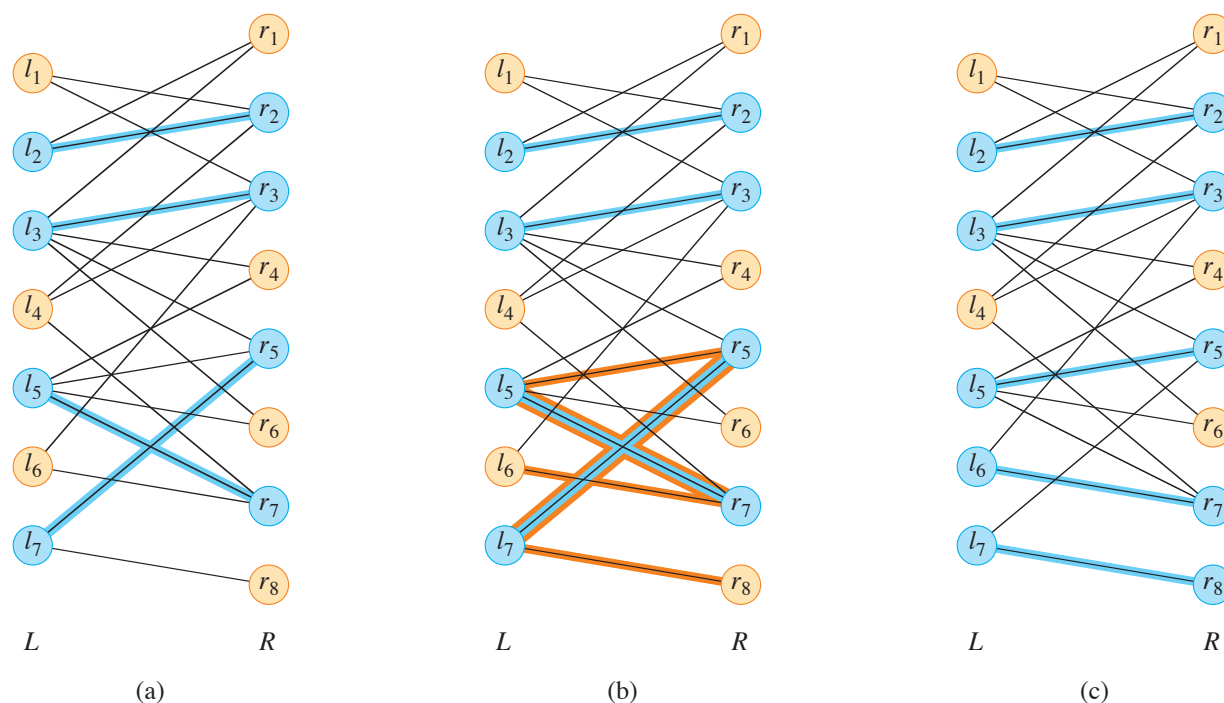


Figure 25.1 A bipartite graph, where $V = L \cup R$, $L = \{l_1, l_2, \dots, l_7\}$, and $R = \{r_1, r_2, \dots, r_8\}$. (a) A matching M with cardinality 4, highlighted in blue. Matched vertices are blue, and unmatched vertices are tan. (b) The five edges highlighted in orange form an M -augmenting path P going between vertices l_6 and r_8 . (c) The set of edges $M' = M \oplus P$ highlighted in blue is a matching containing one more edge than M and adding l_6 and r_8 to the matched vertices. This matching is not a maximum matching (see Exercise 25.1-1).

Figure 25.1(c) demonstrates the following lemma, which shows that by removing from matching M the edges in an M -augmenting path that belong to M and adding to M the edges in the M -augmenting path that are not in M , the result is a new matching with one more edge than M . Since a matching is a set of edges, the lemma relies on the notion of the *symmetric difference* of two sets: $X \oplus Y = (X - Y) \cup (Y - X)$, that is, the elements that belong to X or Y , but not both. Alternatively, you can think of $X \oplus Y$ as $(X \cup Y) - (X \cap Y)$. The operator $\oplus$ is commutative and associative. Furthermore, $X \oplus X = \emptyset$ and $X \oplus \emptyset = \emptyset \oplus X = X$ for any set X , so that the empty set is the identity for $\oplus$.

Lemma 25.1

Let M be a matching in any undirected graph $G = (V, E)$, and let P be an M -augmenting path. Then the set of edges $M' = M \oplus P$ is also a matching in G with $|M'| = |M| + 1$.

Proof Let P contain q edges, so that $\lceil q/2 \rceil$ edges belong to $E - M$ and $\lfloor q/2 \rfloor$ edges belong to M , and let these q edges be $(v_1, v_2), (v_2, v_3), \dots, (v_q, v_{q+1})$. Because P is an M -augmenting path, vertices v_1 and v_{q+1} are unmatched under M and all other vertices in P are matched. Edges $(v_1, v_2), (v_3, v_4), \dots, (v_q, v_{q+1})$ belong to $E - M$, and edges $(v_2, v_3), (v_4, v_5), \dots, (v_{q-1}, v_q)$ belong to M . The symmetric difference $M' = M \oplus P$ reverses these roles, so that edges $(v_1, v_2), (v_3, v_4), \dots, (v_q, v_{q+1})$ belong to M' and $(v_2, v_3), (v_4, v_5), \dots, (v_{q-1}, v_q)$ belong to $E - M'$. Each vertex $v_1, v_2, \dots, v_q, v_{q+1}$ is matched under M' , which gains one additional edge relative to M , and no other vertices or edges in G are affected by the change from M to M' . Hence, M' is a matching in G , and $|M'| = |M| + 1$. ■

Since taking the symmetric difference of a matching M with an M -augmenting path increases the size of the matching by 1, the following corollary shows that taking the symmetric difference of M with k vertex-disjoint M -augmenting paths increases the size of the matching by k .

Corollary 25.2

Let M be a matching in any undirected graph $G = (V, E)$ and $P_1, P_2, \dots, P_k$ be vertex-disjoint M -augmenting paths. Then the set of edges $M' = M \oplus (P_1 \cup P_2 \cup \dots \cup P_k)$ is a matching in G with $|M'| = |M| + k$.

Proof Since the M -augmenting paths $P_1, P_2, \dots, P_k$ are vertex-disjoint, we have that $P_1 \cup P_2 \cup \dots \cup P_k = P_1 \oplus P_2 \oplus \dots \oplus P_k$. Because the operator $\oplus$ is associative, we have

$$\begin{aligned} M \oplus (P_1 \cup P_2 \cup \dots \cup P_k) &= M \oplus (P_1 \oplus P_2 \oplus \dots \oplus P_k) \\ &= (\dots((M \oplus P_1) \oplus P_2) \oplus \dots \oplus P_{k-1}) \oplus P_k. \end{aligned}$$

A simple induction on i using Lemma 25.1 shows that $M \oplus (P_1 \cup P_2 \cup \dots \cup P_{i-1})$ is a matching in G containing $|M| + i - 1$ edges and that path P_i is an augmenting path with respect to $M \oplus (P_1 \cup P_2 \cup \dots \cup P_{i-1})$. Each of these augmenting paths increases the size of the matching by 1, and so $|M'| = |M| + k$. ■

As the Hopcroft-Karp algorithm goes from matching to matching, it will be useful to consider the symmetric difference between two matchings.

Lemma 25.3

Let M and M^* be matchings in graph $G = (V, E)$, and consider the graph $G' = (V, E')$, where $E' = M \oplus M^*$. Then, G' is a disjoint union of simple paths, simple cycles, and/or isolated vertices. The edges in each such simple path or simple cycle

alternate between M and M^* . If $|M^*| > |M|$, then G' contains at least $|M^*| - |M|$ vertex-disjoint M -augmenting paths.

Proof Each vertex in G' has degree 0, 1, or 2, since at most two edges of E' can be incident on a vertex: at most one edge from M and at most one edge from M^* . Therefore, each connected component of G' is either a singleton vertex, an even-length simple cycle with edges alternately in M and M^* , or a simple path with edges alternately in M and M^* . Since

$$\begin{aligned} E' &= M \oplus M^* \\ &= (M \cup M^*) - (M \cap M^*) \end{aligned}$$

and $|M^*| > |M|$, the edge set E' must contain $|M^*| - |M|$ more edges from M^* than from M . Because each cycle in G' has an even number of edges drawn alternately from M and M^* , each cycle has an equal number of edges from M and M^* . Therefore, the simple paths in G' account for there being $|M^*| - |M|$ more edges from M^* than M . Each path containing a different number of edges from M and M^* either starts and ends with edges from M , containing one more edge from M than from M^* , or starts and ends with edges from M^* , containing one more edge from M^* than from M . Because E' contains $|M^*| - |M|$ more edges from M^* than from M , there are at least $|M^*| - |M|$ paths of the latter type, and each one is an M -augmenting path. Because each vertex has at most two incident edges from E' , these paths must be vertex-disjoint. ■

If an algorithm finds a maximum matching by incrementally increasing the size of the matching, how does it determine when to stop? The following corollary gives the answer: when there are no augmenting paths.

Corollary 25.4

Matching M in graph $G = (V, E)$ is a maximum matching if and only if G contains no M -augmenting path.

Proof We prove the contrapositive of both directions of the lemma statement. The contrapositive of the forward direction is straightforward. If there is an M -augmenting path P in G , then by Lemma 25.1, the matching $M \oplus P$ contains one more edge than M , meaning that M could not be a maximum matching.

To show the contrapositive of the backward direction—if M is not a maximum matching, then G contains an M -augmenting path—let M^* be a maximum matching in Lemma 25.3, so that $|M^*| > |M|$. Then G contains at least $|M^*| - |M| > 0$ vertex-disjoint M -augmenting paths. ■